constitution/scripts/multitrack — generic multi-track development engine

Revision: 2 **Last modified:** 2026-07-03T23:20:00Z **Status:** active **Classification:** universal (§11.4.17) — project-agnostic engine; zero project-specific literals (no consumer hardware / vendor / host / package names anywhere in this directory, per §11.4.28(B)).

Project-agnostic runtime for the §11.4.176 / §11.4.167 multi-track development system: worktree-per-alias resolution, the exactly-once claim registry (§11.4.176(A)), the capability-aware deadlock-proof device-lock (§11.4.176(B)), the orchestrator bind/fallback/cooldown lease, the cwd-hook contract, the single-builder rebuild queue, and the tmux launch entrypoint. Every consuming project inherits this directory verbatim (§11.4.28(B) decoupling, §11.4.35 canonical-root) and supplies ONLY its own per-host config data.

What lives here (GENERIC) vs what the consumer supplies (PROJECT DATA)

File	Role
`multitrack_config.sh`	SOURCED library: per-host YAML loader + live-drive detect (by serial, §11.4.111) + protect-guard + track-plan. Provides `mt_repo_root` / `mt_config_dir` / `mt_config_file` / `mt_resolve_host` / `mt_load_config` / `mt_load_pool` + the scalar/list accessors `mt_config_conductor` / `mt_config_fallback_signatures`.
`multitrack.sh`	Entrypoint (`status` / `up`).
`multitrack_resolve_worktree.sh`	Alias → worktree resolution (`resolve`), alias → track resolution (`track` , used by bind-on-start), and the `map` table. Honors the `conductor:` key (conductor stays on /home), orchestrator binding (most-recent wins), else positional map.
`multitrack_registry.sh`	flock TSV get/set over `<repo>/.ws_state/streams.tsv` .
`multitrack_claim.sh`	Exactly-once item→track claim + TTL reap (§11.4.176(A)).
`multitrack_device_lock.sh`	Capability-aware deadlock-proof device-lock (§11.4.176(B)).
`multitrack_cwd_hook.sh`	Thin toolkit adapter: resolves WHICH consumer project this invocation is for (§11.4.177, see below), prints the alias's bound worktree AND fires a best-effort, non-fatal, fully-detached `orchestrator bind` (bind-on-start, §11.4.177), fallback monitor, AND constitution auto-sync for the resolved track (never for the conductor). `--consumer-root [alias]` reports the resolved consumer + why.

File	Role
`multi-track_constitution_sync.sh`	§11.4.35 constitution auto-sync: on every track activation, ancestor-guarded **fast-forward-only** advance of the activated worktree's `constitution/` submodule to latest `<remote>/main` (WIP-preserving, never force/reset/rewind; never `git submodule update`). Keeps the constitution "always up to date with main, Everywhere!"
`multi-track_alias_orchestrator.sh`	Alias↔track bind / fallback / cooldown lease.
`multitrack_build.sh`	Single-builder FIFO rebuild queue.

The consumer supplies its per-host config file (the ONLY place project strings live): track→mount paths, project directory name, device serials + capabilities, the alias↔account map, priority/feature list. In the reference consumer that file is `config/multitrack/<hostname>.yaml`.

11 4 177

Which consumer project? — the cwd-hook's consumer-root resolution (§ . .)

Every engine entry point EXCEPT the cwd-hook runs from INSIDE a consumer checkout, so self-location (below) is the right answer for them. The cwd-hook is the exception: the toolkit invokes it only when cwd is **not** a git repo, and it is customarily installed **once** on a shared PATH. Self-location there answers a question it cannot know — it returns whichever checkout physically holds the engine, shadowing every sibling consumer on the host that ships its own `config/multitrack/`.

`multitrack_cwd_hook.sh` therefore resolves the consumer root itself, first hit wins, and exports it as `MT_REPO_ROOT` so the resolver's existing pin honours it:

#	Source	Notes
1	`MT_REPO_ROOT` env	explicit pin (operator / launcher / test seam)
2	invocation context	nearest ancestor of `$PWD` carrying `config/multitrack/`
3	operator binding file	`MT_CONSUMER_ROOTS`, default `${XDG_CONFIG_HOME:-$HOME/.config}/multitrack/consumer_roots.conf`
4	*(nothing resolved)*	export nothing → resolver keeps its unchanged self-location behaviour

Step 4 is the load-bearing back-compat guarantee: a host with no binding file and no cwd context behaves **exactly** as before this resolution existed.

Honest boundary (§11.4.6). When the hook fires there is, by construction, no cwd project context — so on a host running more than one consumer the alias→consumer mapping is host policy that nothing in-tree can derive. Step 3's operator-owned file is the minimal artifact that satisfies §11.4.177 (project-agnostic tooling) and §11.4.187 (automatic, out-of-the-box) at once.

Binding-file format — parsed, never sourced, so a typo can never execute:

```
# ${XDG_CONFIG_HOME:-$HOME/.config}/multitrack/consumer_roots.conf
default  = /mnt/track1/my_project
claude2  = /mnt/track1/other_project     # alias key beats `default`
```

A named path that does not carry `config/multitrack/` is reported on stderr and **ignored** — never silently trusted (§11.4.6). Diagnose with:

```
multitrack_cwd_hook.sh --consumer-root <alias>
```

> **Installation note (§11.4.177).** Resolution being project-agnostic is only half the rule. The shared PATH entry itself must not live inside a consumer checkout — a symlink from `~/.local/bin/` into one project's tree couples every project's tooling availability and version to that one checkout. Install the engine from a location outside every consumer, or use the toolkit's per-project `<dir>/.claude-cwd-hook` / `CMA_CWD_HOOK` seams.

Config contract — how a generic script finds project config

A generic script self-locates BOTH roots with zero hardcoding:

- **Constitution root** — via `constitution/find_constitution.sh` (honors an explicit `CONSTITUTION_DIR` override, else walks up for `constitution/Constitution.md` / `submodules/constitution/Constitution.md`, else follows the git superproject pointer). These scripts do not need it at runtime (they are self-contained), but it is the canonical self-location primitive when a caller needs the constitution root.
- **Project/repo root** — `MT_REPO_ROOT` (explicit override) else the script's `../..`. **Because these scripts live at `constitution/scripts/multitrack/`, their `../..` resolves to the constitution submodule, NOT the consuming project.** A consumer therefore MUST set `MT_REPO_ROOT` (or the explicit `MT_CONFIG` full path) so config, `.ws_state`, and the tmp namespace resolve against the project, not the constitution. The reference launcher exports `MT_REPO_ROOT` for every invocation.
- **Per-host config file** — resolved by `multitrack_config.sh` as:
  1. `MT_CONFIG` — explicit full path to a config file (wins), else
  2. `$MT_CONFIG_DIR/<host>.yaml` — where `MT_CONFIG_DIR` defaults to `$(mt_repo_root)/config/multitrack` and `<host>` = `MT_HOST` else the system hostname (`/etc/hostname`, else the first 16 of the machine-id). An unreadable `<hostname>.yaml` falls back to `<machine-id-16>.yaml`.

Schema: `schema_version: 1`, **one file per host**.

The `MT_*` environment contract (retained verbatim, no renames)

Env	Meaning	Default
`MT_REPO_ROOT`	project/repo root	script `../..` (⇒ set this from the constitution location)
`MT_CONFIG`	explicit config-file path (overrides host resolution)	—
`MT_CONFIG_DIR`	per-host config directory	`$(mt_repo_root)/config/multitrack`
`MT_HOST`	host key for `<host>.yaml`	system hostname → machine-id-16
`MT_ALIAS_DIR`	orchestrator runtime dir (bindings/cooldowns/events)	`${XDG_RUNTIME_DIR:-/tmp}/$(basename "$MT_REPO_ROOT")/multitrack/aliasorch`
`MT_LOCK_DIR`	device-lock runtime dir	`${XDG_RUNTIME_DIR:-/tmp}/$(basename "$MT_REPO_ROOT")/multitrack/devicelock`
`MT_REGISTRY_LOCK`	streams-registry flock	`${XDG_RUNTIME_DIR:-/tmp}/$(basename "$MT_REPO_ROOT")/multitrack/registry.lock`
`MT_STREAMS_TSV`	streams registry TSV	`$MT_REPO_ROOT/.ws_state/streams.tsv`
`MT_WS_STATE`	per-repo state dir	`$MT_REPO_ROOT/.ws_state`
`MT_WORKTREE_SUBDIR`	worktree subdir under a track mount	`$(basename "$MT_REPO_ROOT")`
`MT_ALIAS_ROSTER`	alias roster override (`name:kind,...`)	config `aliases:` block, else built-in

Env	Meaning	Default
`MT_BUILD_COMMAND`	build command string (the `build.command` config key)	auto-detect `scripts/build_maxres.sh` else `scripts/build.sh`
`MT_BUILD_MIN_FREE_GB`	pre-build free-space floor (the `build.min_free_gb` config key)	`158`
`MT_LOCK_WAIT` / `MT_DEFAULT_TTL` / `MT_CLAIM_TTL` / `MT_ALIAS_TTL` / `MT_COOLDOWN`	flock/lease timers	see each script header

No `MT_*` var was renamed when this engine was lifted from the reference consumer — a rename would break the live host and every paired test (§11.4.6).

Consumer config keys (in `config/multitrack/<host>.yaml`)

Beyond `host` / `protected_drives` / `tracks` / `device_pool` / `lease_policy` / `aliases`, the config carries two OPTIONAL keys read by the engine. Both are **consumer data** (§11.4.28(B)) — the engine never hardcodes their values.

Key	Shape	Read by	Meaning
`conductor:`	top-level scalar (alias name)	`mt_config_conductor` → the resolver	§11.4.177 auto-conductor. The named alias is NEVER worktree-bound — its session stays on the shared `/home` checkout (`resolve` prints nothing / exit 0, and bind-on-start never binds it). **Empty string (default) ⇒ no alias is special-cased** (the `/home` session is simply whichever session is unbound). Consumer-overridable (e.g. `conductor: claude3` , or a provider alias to dedicate a conductor account).
`fallback.signatures:`	block-style list under a top-level `fallback:`	`mt_config_fallback_signatures` → the (PWU-4) rate-limit monitor	§11.4.177 auto-fallback. The rate-limit transcript signatures the monitor matches to detect a usage/quota limit and trigger `orchestrator fallback` . Surrounding single/double quotes are stripped (values that themselves contain `"` are single-quoted in YAML). **Consumer-overridable** — a future rate-limit wording change is a one-line pin here, not a code change. Empty/absent ⇒ no signatures (the monitor decides its own default).

Reference-consumer seed (both keys are consumer data, so they live in the project config, never in this engine):

```yaml
conductor: ""          # empty ⇒ /home session is conductor
fallback:
  signatures:
    - '"apiErrorStatus":429'
    - '"isApiErrorMessage":true'
```

Runtime namespace derivation

The tmp runtime dirs (`aliasorch` / `devicelock` / `registry.lock`) derive their top-level namespace from `$(basename "$MT_REPO_ROOT")` (via `mt_repo_root`), so a consumer's runtime state is namespaced by its own project directory name — no project literal is baked into the engine.

Quick start (consumer)

```
# from the project root, with a config/multitrack/<hostname>.yaml present:
MT_REPO_ROOT="$PWD" bash constitution/scripts/multitrack/multitrack_resolve_worktree.sh map
# or point at an explicit config file:
MT_CONFIG=config/multitrack/<host>.yaml MT_REPO_ROOT="$PWD" \
  bash constitution/scripts/multitrack/multitrack_resolve_worktree.sh resolve claude1
```

11 4 187

Default single-track mode — multitrack works on EVERY host (§ . .)

You do not need a per-host config to use this engine. A host with no `config/multitrack/<hostname>.yaml` runs in **default single-track mode**:

tracks	exactly **one**
track-1	the **invocation project root**, used as-is
worktrees	none — the project root *is* the track
drives	none — track-1 is a plain directory, not a mounted volume
notice	printed loudly on stderr on every activation

Multiple tracks are opt-in: you add them **only when explicitly required**, by authoring the per-host config below.

Why this is not a § . . violation

Host *data* — mount paths, drive serials, alias→track maps — genuinely cannot be guessed, and the engine still refuses to invent any of it. But *"Track 1 is the project root you are standing in"* is not invented data: it is a **defined default**, derivable with certainty from the invocation context (§11.4.177 invocation-directory operation). Fataling when a correct and safe default exists is the §11.4.201 false-refusal class — which is what the engine used to do, and what made multi-track work on exactly one hand-provisioned host.

The three cases, kept strictly apart

situation	behaviour
a host config **exists**	loaded exactly as before — **no behaviour change at all**
a host config **is malformed** (unparsable, or zero tracks)	**still FATAL.** That *is* ambiguity and is never defaulted past (§11.4.6)
no host config at all	default single-track mode + a loud notice

A defaulted setup can never be silently mistaken for a provisioned one: the notice names the[1] mode, the host, the config path it looked for, the track count, and the resolved Track 1.

Track does not have to be the project root

It is only the **default**. Point Track 1 anywhere — `/mnt/track-1`, a dedicated drive, a worktree — by declaring it explicitly in the per-host config. The default exists so that a host with nothing configured still works, not to constrain a host that is configured.

Overrides

variable	effect
`MT_REQUIRE_HOST_CONFIG=1`	strict mode: restore the pre-§11.4.187 behaviour — a missing host config is fatal, never defaulted
`MT_REPO_ROOT=/path`	pin the project root explicitly instead of deriving it from the invocation context
`MT_CONFIG=/path/to.yaml`	use an explicit config file, bypassing the hostname-keyed lookup

If the project root cannot be resolved at all, the engine **fails loudly** — it never silently defaults Track 1 to a wrong location.

Inspecting activation before it touches anything

```
# report only: installs no symlink, starts no daemon, reconciles nothing
bash constitution/scripts/multitrack/multitrack_bootstrap.sh --check
```

`--check` (alias `--dry-run`) prints each step's decision and what it *would* run. Two consecutive `--check` runs produce byte-identical output.

Directory tracks vs drive tracks

A track declared **without** a `drive_serial` is a *directory track*: it is READY when its directory exists. The engine never probes `/proc/mounts` for one, so a directory track is never reported as "unmounted" and never drags in the LUKS/mount operator hand-off that does not apply to it. A track **with** a `drive_serial` behaves exactly as before.

Likewise, a host that declares no `device_pool` reports an honestly empty pool instead of failing; only the commands that genuinely need a device (`acquire` / `release` / `heartbeat` / `reap` / `reconcile`) refuse.

Alias mapping in single-track mode

Aliases are mapped positionally onto **feature**-role tracks. A single main-only track has no feature tracks, so no alias is worktree-bound — every session simply stays on the one checkout, which is the project root. That is the correct outcome, and `map` reports it as `fallback(/home)` for each alias.

Genericization delta vs the reference consumer

Lifted from the reference project's `scripts/multitrack/` per §11.4.35 with a small, mechanical delta (three executed-code fixes + comment-literal scrub):

1. **Namespace literal** — the tmp-namespace defaults in `multitrack_registry.sh`, `multitrack_resolve_worktree.sh`, `multitrack_device_lock.sh`, and `multitrack_alias_orchestrator.sh` derive the top-level dir from `$(basename "$MT_REPO_ROOT")` instead of a hardcoded project name (orchestrator + resolve share one runtime dir, so both are fixed in lockstep).
2. **Build command** — `multitrack_build.sh` reads the `build.command` config key via `MT_BUILD_COMMAND`, falling back to a documented auto-detect default.
3. **Item-id param** — `multitrack_claim.sh`'s user-facing `atm-id` label is the scheme-agnostic `item-id` (the positional arg accepts any stable id — no named flag existed, so no back-compat alias is needed).

Consumer-side DATA (`config/multitrack/<host>.yaml`, serials, mount paths) stays in the consuming project and is NEVER copied here.

11 4 196

Alias priority, per-alias limit/subscription tracking, auto-rebind-on-recovery (§ . .)

The alias↔track binding layer (`multitrack_alias_orchestrator.sh`) implements the §11.4.196 mandate — the alias-binding + rate-limit/subscription-tracking layer of the §11.4.187 ruler:

Native-alias-first priority. `_next_available_alias` scans the `native` class COMPLETELY before the `provider` class, so an OPERATIONAL native (`claudeN`) alias is ALWAYS selected before ANY provider, regardless of roster file order (§11.4.111 resolve-by-CLASS-not-by-file-position). Within a class the operator-mandated order (native equal-capability; providers `deepseek → xiaomi → opencode → kimi-for-coding → …`) is authoritative decision-DATA — the built-in `DEFAULT_ROSTER` encodes it and `config/multitrack/alias_priority.yaml` is its tracked, non-secret record (alias NAMES only, never keys — §11.4.10). `_alias_rank` exposes the same ordering as a comparable integer (native base 0 < provider base 100000).

Per-alias reason-class limit tracking (from REAL signals, never faked — §11.4.6). A cooled alias's `cooldowns.snapshot` row is `alias|until|reason|class` where `class ∈ {session | weekly | subscription}` and `until` is its operational-again epoch. The three classes have DIFFERENT windows:

class	window (`MT_COOLDOWN_*`)	meaning
`session`	`MT_COOLDOWN_SESSION` (=300s)	a 429 session rate-limit — self-heals at the reset
`weekly`	`MT_COOLDOWN_WEEKLY` (=604800s)	weekly-limit-reached — ~until the weekly reset
`subscrip-tion`	far-future sentinel (2100-01-01Z)	subscription expired — INDEFINITE until a REAL renewal

`§11.4.6` NEVER guesses a renewal date — a caller MAY pass an exact `--until <epoch>` parsed from a real reset marker, which wins over the class default.

Commands.

- `mark-limited --alias A [--class session|weekly|subscription] [--until EPOCH] [--reason R]` — record a per-alias limit from a real signal (the `multitrack_fallback_monitor.sh` 429-signature classifier calls this / `fallback --class …`). A limited alias is never auto-selected until `until` passes (session/weekly) or `mark-operational` clears it (subscription).

- `mark-operational --alias A` — clear an alias's cooldown NOW (a subscription renewal / operator-confirmed recovery). The next `auto-assign` / `promote` prefers it.
- `promote [--ttl S]` — auto-use-on-recovery: for EVERY bound track, if a strictly higher-priority (lower `_alias_rank`) NON-cooled alias exists, rebind the track UPWARD to it, PRESERVING the worktree AND device leases (leases key on the STABLE track id — §11.4.119). Idempotent. Run it after every reap/tick so a track that fell onto a provider returns to a recovered native automatically.
- `fallback --track T [--class …] [--until …]` — on a limit, cool the current alias (class-aware) and hand the track to the next available alias.
- `status` — the cooldown section now shows `class` + `operational-again-epoch`.

RB-02 host-budget guard fix (`multitrack_host_budget.sh`, §12.12 / §11.4.196(D)). `_mt_host_budget_heavy_build_running` no longer trusts a bare `pgrep -f REGEX` substring match (which false-matched a live `claude` worker whose prompt QUOTED the pgrep pattern and REFUSED every spawn on a host with zero real builds — forensic FACT 2026-07-13). It re-reads each matched PID's REAL `/proc/<pid>/cmdline` (via the `_mt_host_budget_pid_cmdline` seam) and excludes the pattern-CARRIER / multitrack- ENGINE / `claude`-worker / self+parent classes; an UNREADABLE cmdline is conservatively counted as a build (REFUSE — the safe, reversible default, §11.4.101/§12.8), so a genuine soong/gradle/JVM-daemon build is still caught.

Tests (anti-bluff, §11.4.115 RED-polarity + §1.1 mutation + §11.4.50 determinism):

- `test_multitrack_host_budget_rb02_pgrep.sh` — RB-02 guard (hermetic stub + real-`/proc` E2E).
- `test_multitrack_alias_priority_limits.sh` — native-first rank + per-class limit windows + `promote`.
- `test_multitrack_native_first_fallback.sh` — native-first selection (pre-existing). Run each with `RED_MODE=1` (reproduce the defect on the mutated artifact) and `RED_MODE=0` (assert the fixed behaviour + §1.1 self-check).

Honest boundary (§11.4.6): these prove the selection / tracking / rebind / guard CONTRACT on the mechanism; genuine live per-subscription quota-isolation still needs live tokens exercising real rate-limit boundaries (an operator acceptance window).